

TECHNICAL TRAINING SERIES

LINUX

Training Documentation

01
FOUNDATIONS

02
COMMAND LINE

03
SYSTEM ADMIN

04
SHELL SCRIPTING

An overview of the origin, architecture, and fundamentals of the Linux operating system, along with its core commands and concepts.

A practical reference for students and beginners

A structured route through the Linux fundamentals covered in this training guide.

01 Introduction to Linux	02 Origin and History of Linux
03 Linux Architecture	04 Features of Linux
05 Linux Distributions	06 The Linux File System
07 Basic Linux Commands	08 File and Directory Permissions
09 Users and Groups	10 Process Management
11 Package Management	12 Redirection, Pipes, and I/O
13 Shell Scripting Basics	14 Summary

DOCUMENT MAP

Concepts → file system → commands → permissions → processes → packages → I/O → scripting

What Linux is, why it is learned, and where it is used.

1.1 What is Linux?

- Linux is a free, open-source operating system kernel that forms the core of many complete operating systems, commonly called Linux distributions.
- It manages a computer's hardware resources — CPU, memory, storage, and devices — and allows software to run on top of it.
- It is widely used across servers, desktops, embedded systems, mobile devices (Android), and supercomputers.

1.2 Why Learn Linux?

- Powers the majority of the world's servers, cloud infrastructure, and websites.
- Open-source and free to use, study, modify, and distribute.
- Highly stable, secure, and customizable compared to many other operating systems.
- Essential skill for developers, system administrators, and DevOps engineers.

1.3 Where Is Linux Used?

Servers	Web servers, database servers, and cloud infrastructure
Desktops	Everyday computing through distributions like Ubuntu, Fedora, and Linux Mint
Mobile Devices	Android is built on the Linux kernel
Embedded Systems	Routers, smart TVs, IoT devices, and automotive systems
Cloud & Containers	Cloud instances and container platforms such as Docker
Supercomputers	Widely used for high-performance computing

From Unix and GNU to the Linux kernel.

2.1 The Unix Background

- In the late 1960s and 1970s, Unix was developed at Bell Labs by Ken Thompson and Dennis Ritchie.
- Unix introduced key ideas still used today — a hierarchical file system, multitasking, and multi-user support.
- Unix was proprietary, which encouraged the development of free alternatives.

2.2 The GNU Project

- In 1983, Richard Stallman started the GNU Project to build a completely free Unix-like operating system.
- The GNU Project produced many essential tools — compilers, editors, and utilities — but initially lacked a working kernel.

2.3 The Birth of Linux

- In 1991, Linus Torvalds, a Finnish computer science student, began developing a free kernel as a personal project.
- He announced it on the comp.os.minix newsgroup as a hobby project.
- The Linux kernel was combined with GNU tools to form complete usable operating systems.
- Released under the GNU General Public License (GPL), allowing the source code to be used, studied, modified, and shared.

DID YOU KNOW

Linux is named after its creator, Linus Torvalds, combined with Unix. The Linux mascot is a penguin named Tux.

2.4 Timeline at a Glance

1969–1970	Unix developed at Bell Labs
1983	Richard Stallman launches the GNU Project
1991	Linus Torvalds releases the first Linux kernel (v0.01)
1992	Linux is relicensed under the GNU GPL
1994	Linux kernel version 1.0 released
1996 onward	Distributions like Red Hat, Debian, and SUSE gain popularity
2004	Ubuntu released
Today	Linux powers servers, cloud platforms, and Android devices

The major layers and responsibilities inside a Linux system.

2.5 Key People Behind Linux

Ken Thompson & Dennis Ritchie	Created Unix at Bell Labs, laying the conceptual foundation for Linux
Richard Stallman	Founded the GNU Project and the Free Software Foundation
Linus Torvalds	Created the Linux kernel and continues to lead its development
Open-Source Community	Thousands of contributors worldwide maintain and improve the kernel and tools

Architecture Layers

Hardware	CPU, RAM, disk, network devices
Kernel	Core of the OS; manages hardware resources, memory, processes and system calls
Shell	Command interpreter that lets users interact with the kernel
System Utilities / Applications	Editors, browsers, compilers and other user programs

3.1 The Kernel's Core Responsibilities

- Process management — scheduling and controlling running programs.
- Memory management — allocating and freeing memory.
- Device management — communicating with hardware through drivers.
- File system management — organizing and accessing data on storage devices.

How applications request services from the kernel.

3.2 System Calls

Applications do not talk to hardware directly. Instead, they request services from the kernel through system calls — a defined interface for operations like opening a file, creating a process, or sending data over a network. This separation keeps the system stable and secure.

4. Features of Linux

Open Source	Source code is freely available to view, modify, and distribute.
Multiuser	Multiple users can access the system and run programs simultaneously.
Multitasking	Can run several processes at the same time.
Portability	Runs on a wide range of hardware.
Security	Strong permission and user-privilege model reduces unauthorized access.
Stability	Known for long uptimes and reliability.
Community Support	A large global community continuously improves and supports the ecosystem.

5. Linux Distributions

A Linux distribution bundles the Linux kernel with system tools, package managers, and software to create a complete, ready-to-use operating system.

Ubuntu	Desktops, beginners, and general-purpose servers
Debian	Stability-focused servers; base for many other distros
Fedora	Cutting-edge features, desktop use, and development
RHEL	Enterprise servers and business-critical systems
CentOS / Rocky / AlmaLinux	Free, RHEL-compatible server operating systems
Arch Linux	Highly customizable, rolling-release systems

Choosing a distribution and understanding the file system.

5.1 Choosing the Right Distribution

- Beginners — Ubuntu or Linux Mint, for ease of setup and large community support.
- Servers — Debian, Ubuntu Server, or RHEL/Rocky Linux, for stability and long-term support.
- Advanced users — Arch Linux, for full control over installed components.
- Security professionals — Kali Linux, which ships with pre-installed security and testing tools.

Kali Linux	Security testing and penetration testing
------------	--

6. The Linux File System

6.1 Structure

- Linux organizes all files and directories in a single hierarchical tree, starting from the root directory, written as `/`.
- Unlike Windows, there are no separate drive letters — all storage devices are mounted into this one tree.

6.2 Key Directories

<code>/</code>	The root of the entire file system
<code>/home</code>	Personal directories for each user
<code>/etc</code>	System-wide configuration files
<code>/bin</code>	Essential user command binaries
<code>/var</code>	Variable data — logs, caches, spool files
<code>/tmp</code>	Temporary files
<code>/usr</code>	User-installed software and libraries
<code>/root</code>	Home directory of the root administrator
<code>/dev</code>	Device files representing hardware components

Paths and the first group of everyday terminal commands.

6.3 Absolute vs. Relative Paths

- An absolute path starts from the root directory (/) and specifies the full location of a file.
- A relative path is based on the current working directory and is often shorter to type.

```
# Absolute path
$ cd /home/student/Documents

# Relative path
$ cd Documents

# Shortcuts
$ cd ..
$ cd ~
$ cd .
```

7. Basic Linux Commands

7.1 Navigation

pwd	Print the current working directory
ls	List files and directories
cd	Change the current directory
tree	Display directories as a tree structure
<pre>\$ pwd /home/student \$ ls -l \$ cd Documents</pre>	

7.2 File and Directory Operations

touch file.txt	Create an empty file
mkdir folder	Create a new directory
cp source dest	Copy a file or directory

Viewing, editing, system information, and searching.

7.2 File and Directory Operations

<code>mv source dest</code>	Move or rename a file or directory
<code>rm file.txt</code>	Delete a file
<code>rm -r folder</code>	Delete a directory and its contents

7.3 Viewing and Editing Files

<code>cat file.txt</code>	Display the contents of a file
<code>less file.txt</code>	View a file page by page
<code>nano file.txt</code>	Edit a file using nano
<code>vi file.txt / vim file.txt</code>	Edit a file using vi/vim

7.4 System Information

<code>whoami</code>	Show the current logged-in user
<code>uname -a</code>	Display system and kernel information
<code>df -h</code>	Show disk space usage
<code>free -h</code>	Show memory usage

7.5 Searching for Files and Text

<code>find /path -name file.txt</code>	Search for files by name
<code>grep "text" file.txt</code>	Search for a pattern inside a file
<code>grep -r "text" .</code>	Search recursively through files
<code>locate file.txt</code>	Quickly find a file using a system index

```
$ find /home -name "*.log"
$ grep -r "error" /var/log
```

Compression, archives, and network utilities.

7.6 Compression and Archives

<code>tar -cvf archive.tar folder/</code>	Create an archive
<code>tar -xvf archive.tar</code>	Extract files from a tar archive
<code>tar -czvf archive.tar.gz folder/</code>	Create a compressed gzip archive
<code>zip -r archive.zip folder/</code>	Create a ZIP archive
<code>unzip archive.zip</code>	Extract a ZIP archive

7.7 Basic Networking Commands

<code>ping google.com</code>	Test connectivity to a remote host
<code>ip a / ifconfig</code>	Show network interfaces and IP addresses
<code>curl url</code>	Fetch contents of a URL
<code>wget url</code>	Download a file from the internet
<code>ssh user@host</code>	Connect securely to a remote machine

8. File and Directory Permissions

8.1 Permission Types

- Read (r) — view the contents of a file or list a directory.
- Write (w) — modify a file, or add/remove files within a directory.
- Execute (x) — run a file as a program, or enter a directory.

8.2 Permission Groups

- Owner — the user who owns the file.
- Group — users who belong to the file's assigned group.
- Others — everyone else on the system.

Permission strings, chmod, ownership, and special permissions.

```
$ ls -l notes.txt
-rw-r--r-- 1 student staff 1024 Sep 27 10:00 notes.txt

$ chmod 755 script.sh
$ chown student:staff notes.txt
```

Reading Permission Strings

In **-rw-r--r--**, the first character indicates the file type. The next three groups of three characters represent owner, group, and others' permissions respectively.

8.3 Numeric (Octal) Permissions

Read = 4, write = 2, execute = 1. These values are added to represent permissions for owner, group, and others.

7	Read + Write + Execute (rwx)
6	Read + Write (rw-)
5	Read + Execute (r-x)
4	Read only (r--)
0	No permissions (---)

```
# rwxr-xr-- => owner: 7, group: 5, others: 4
$ chmod 754 script.sh
```

8.4 Special Permissions

- SetUID — a file runs with the permissions of its owner.
- SetGID — a file or new files in a directory inherit the group of the owning group.
- Sticky Bit — in a shared directory, only the file's owner can delete or rename it.

9. Users and Groups

- Linux is a multiuser system — every user has a unique username and user ID (UID).
- Users can belong to one or more groups, simplifying shared permissions.

Accounts, groups, root privileges, and important system files.

- The root user (or superuser) has full administrative privileges over the system.

adduser username	Create a new user account
passwd username	Set or change a user's password
groups username	Show the groups a user belongs to
sudo command	Run a command with administrative privileges

9.1 Important User-Related Files

/etc/passwd	Stores basic information about each user account
/etc/shadow	Stores encrypted user passwords, readable only by root
/etc/group	Defines groups and their member users

9.2 su vs. sudo

- su — switches to another user account entirely, commonly the root user.
- sudo — runs a single command with elevated privileges, requiring the current user's authorization.
- sudo is generally preferred because it logs actions and avoids staying logged in as root.

10. Process Management

Every running program on Linux is represented as a process, identified by a unique Process ID (PID).

ps	List currently running processes
top / htop	Display real-time system resource and process usage
kill PID	Terminate a process by its ID
bg / fg	Send a process to the background or foreground

Processes, jobs, priorities, and package managers.

```
$ ps aux | grep firefox
$ kill 4821
```

10.1 Job Control and Priority

jobs	List background jobs in the current shell session
command &	Run a command in the background
nice -n 10 command	Start a command with a lower scheduling priority
renice 10 -p PID	Change the priority of an already-running process

11. Package Management

Package managers install, update, and remove software along with its dependencies, saving users from manual setup.

Debian / Ubuntu	apt
Red Hat / Fedora / CentOS	dnf or yum
Arch Linux	pacman

```
# Ubuntu / Debian
$ sudo apt update
$ sudo apt install git
```

11.1 Building Software from Source

When a package is not available through a package manager, software can be compiled directly from its source code.

```
$ ./configure
$ make
$ sudo make install
```

Moving command input and output between files and programs.

12.1 Redirection Operators

>	Redirect output to a file, overwriting its contents
>>	Redirect output to a file, appending to existing contents
<	Redirect a file's contents as input to a command
2>	Redirect error messages to a file

```
$ echo "Hello" > greeting.txt
$ echo "World" >> greeting.txt
$ cat < greeting.txt
$ ls /no/such/folder 2> errors.log
```

12.2 Pipes

The pipe operator (|) sends the output of one command directly into another command as input, allowing several commands to be chained together.

```
$ ps aux | grep bash
$ cat access.log | grep "404" | wc -l
```

13. Shell Scripting Basics

A shell script is a text file containing a sequence of commands that the shell executes in order. Scripts are used to automate repetitive tasks.

13.1 A Simple Script

```
#!/bin/bash
# greet.sh - a simple shell script
name="Student"
echo "Hello, $name! Today is $(date)"
```

Making scripts executable and using conditions and loops.

13.2 Making a Script Executable

```
$ chmod +x greet.sh  
$ ./greet.sh
```

13.3 Conditionals and Loops in Bash

```
#!/bin/bash  
count=1  
while [ $count -le 3 ]; do  
  if [ $count -eq 2 ]; then  
    echo "Count is two"  
  else  
    echo "Count is $count"  
  fi  
  count=$((count + 1))  
done
```

14. Summary

Linux began as a personal project by Linus Torvalds in 1991 and, built on the foundations laid by Unix and the GNU Project, has grown into one of the most widely used operating systems in the world. This documentation has covered its origin and history, layered architecture, file system, essential commands, permissions, user management, processes, package management, I/O redirection, and shell scripting.

KEY TAKEAWAY

Understanding Linux's history and fundamentals provides the groundwork for working confidently with servers, cloud environments, and development systems.

END OF DOCUMENT